

Global State



Global state of a distributed system

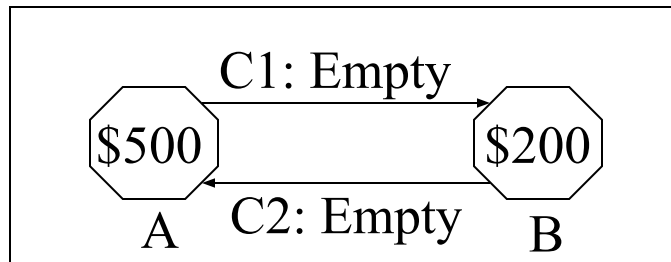
- Absence of global clock leads to a problem in global state recording

What is global state?

- Local state of each process and
- The state of communication channels i.e. messages that are currently in transit (sent but not received)

Global State

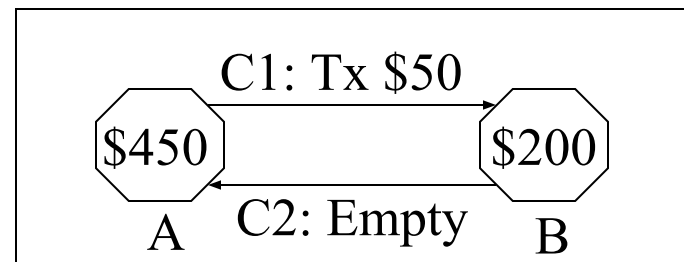
Global State 1



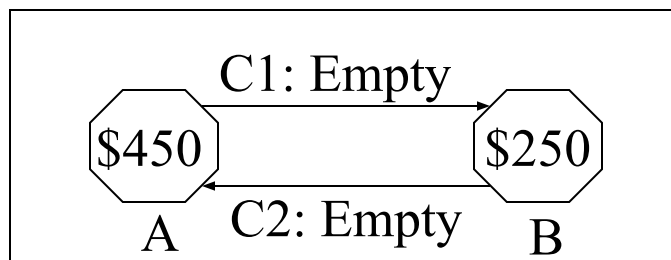
State of C1, C2 and Account B was recorded when the global state was 2

State of Account A at site S1 was recorded when the global state was 1

Global State 2



Global State 3



Composite of all states recorded shows an extra amount of \$50 would appear in the global state. This is due to A's state was recorded before the msg was sent and channel state was recorded after the msg was sent

How to record the global state

□ Distributed snapshot

- reflects a state in which the distributed system ***might*** have been
- reflects a consistent global state
- If we have recorded that process P has received a msg from another process Q, then we should also have recorded that process Q had actually ***sent*** the msg
- The reverse condition (Q has sent a msg that P has not yet received) is allowed.

Recording Global State

- $\text{Send}(M_{ij})$: message M sent from S_i to S_j
- $\text{rec}(M_{ij})$: message M received by S_j , from S_i
- $\text{time}(x)$: Time of event x
- LS_i : local state at S_i
- $\text{send}(M_{ij})$ is in LS_i iff (if and only if)
 $\text{time}(\text{send}(M_{ij})) < \text{time}(\text{LS}_i)$
- $\text{rec}(M_{ij})$ is in LS_j iff $\text{time}(\text{rec}(M_{ij})) < \text{time}(\text{LS}_j)$
- $\text{transit}(\text{LS}_i, \text{LS}_j)$: set of messages
sent/recorded at LS_i and NOT
received/recorded at LS_j

Recording Global State ...

- $\text{inconsistent}(\text{LS}_i, \text{LS}_j)$: set of messages NOT sent/recorded at LS_i and received/recorded at LS_j
- Global State, GS: $\{\text{LS}_1, \text{LS}_2, \dots, \text{LS}_n\}$
- Consistent Global State, GS = $\{\text{LS}_1, \dots, \text{LS}_n\}$
AND for all i in n , $\text{inconsistent}(\text{LS}_i, \text{LS}_j)$ is null.
- Transitless global state, GS = $\{\text{LS}_1, \dots, \text{LS}_n\}$
AND for all i in n , $\text{transit}(\text{LS}_i, \text{LS}_j)$ is null.

Recording Global State ...

For a message m_{ij} sent by S_i to S_j , we say that

- $send(m_{ij}) \in LS_i$ iff $time(send(m_{ij})) < time(LS_i)$.
- $rec(m_{ij}) \in LS_j$ iff $time(rec(m_{ij})) < time(LS_j)$.

For the local states LS_i and LS_j of any two sites S_i and S_j , we define two sets of messages. These sets contain messages that were sent from site S_i to site S_j .

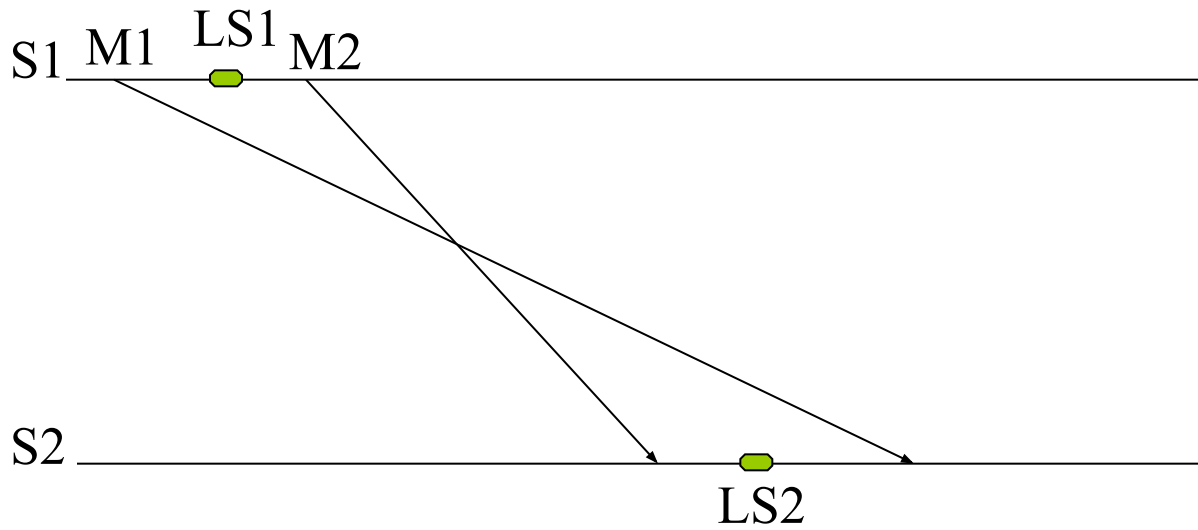
Transit: $transit(LS_i, LS_j) = \{m_{ij} \mid send(m_{ij}) \in LS_i \wedge rec(m_{ij}) \notin LS_j\}$

Inconsistent: $inconsistent(LS_i, LS_j) = \{m_{ij} \mid send(m_{ij}) \notin LS_i \wedge rec(m_{ij}) \in LS_j\}$

GLOBAL STATE. A global state, GS , of a system is a collection of the local states of its sites; That is, $GS = \{LS_1, LS_2, \dots, LS_n\}$ where n is the number of sites in the system.

Note that any collection of local states of sites need not represent a consistent global state. Consistency has a connotation that for every effect or outcome recorded in a global state, the cause of the effect must also be recorded in the global state. We next give definitions characterizing global states.

Recording Global State ..



M1: ?

M2: ?

Recording Global State ...

Consistent global state. A global state $GS = \{LS_1, LS_2, \dots, LS_n\}$ is *consistent* iff

$$\forall i, \forall j : 1 \leq i, j \leq n :: inconsistent(LS_i, LS_j) = \Phi$$

Thus, in a consistent global state, for every received message a corresponding send event is recorded in the global state. In an inconsistent global state, there is at least one message whose receive event is recorded but its send event is not recorded in the global state. In Fig. 5.8, the global state $\{LS_{12}, LS_{23}, LS_{33}\}$ and $\{LS_{11}, LS_{22}, LS_{32}\}$ correspond to consistent and inconsistent global states, respectively.

Transitless global state. A global state is *transitless* if and only if

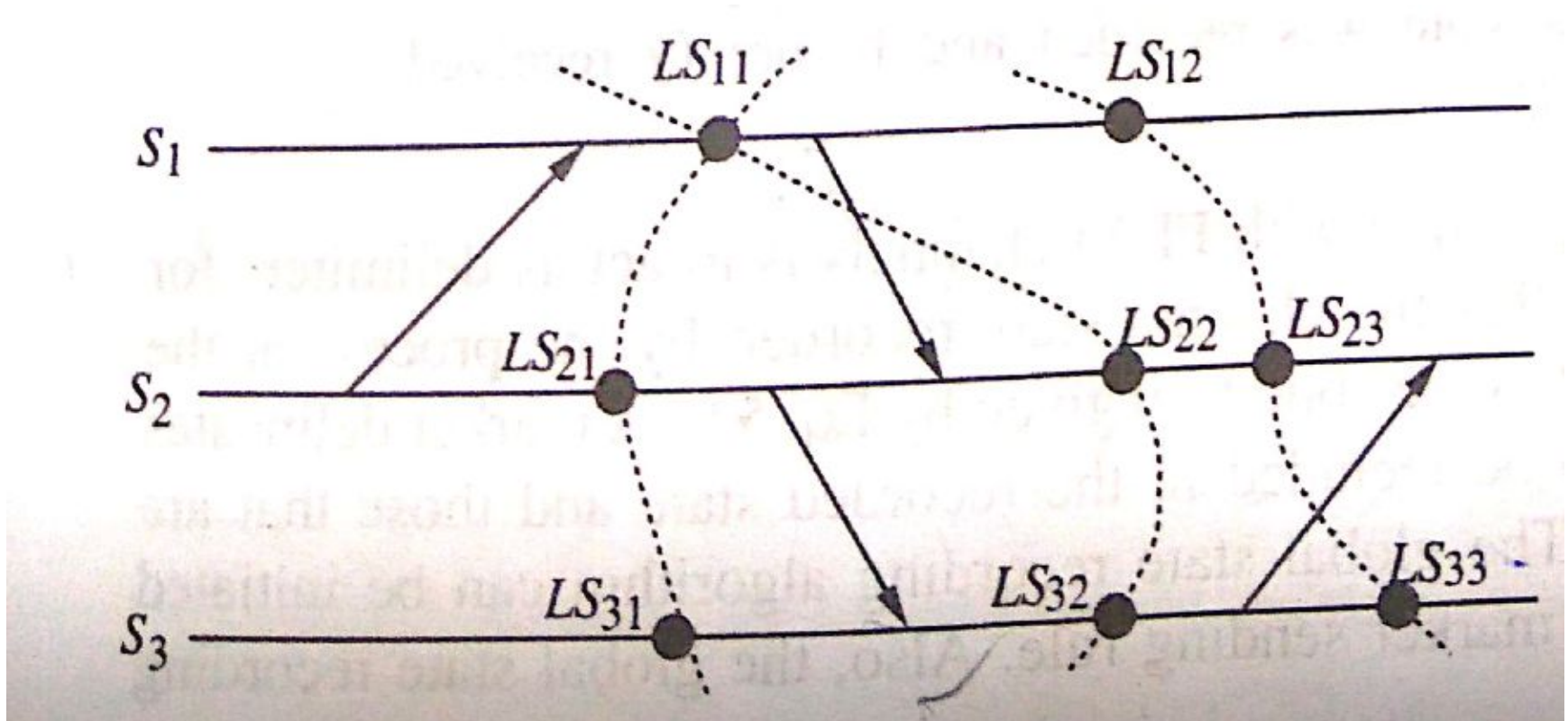
$$\forall i, \forall j : 1 \leq i, j \leq n :: transit(LS_i, LS_j) = \Phi$$

Thus, all communication channels are empty in a transitless global state.

Strongly consistent global state. A global state is *strongly consistent* if it is consistent and transitless. In a strongly consistent state, not only the send events of all the recorded and transitless. In a strongly consistent state, not only the send events of all the recorded received events are recorded, but the receive events of all the recorded send events are also recorded. Thus, a strongly consistent state corresponds to a consistent global state in which all channels are empty. In Fig. 5.8, the global state $\{LS_{11}, LS_{21}, LS_{31}\}$ is a strongly consistent global state.

Example

Identify consistent, inconsistent, and strongly consistent global state.



Assumptions of Chandy-Lamport algorithm

- There are finite number of processes in the distributed system and they do not share memory and clocks.
- There are finite number of communication channels and they are unidirectional and FIFO ordered.
- There exists a communication path between any two processes in the system
- On a channel, messages are received in the same order as they are sent.

Chandy-Lamport algorithm

Chandy-Lamport algorithm

The Chandy-Lamport algorithm uses a control message, called a *marker* whose role in a FIFO system is to separate messages in the channels.

After a site has recorded its snapshot, it sends a *marker*, along all of its outgoing channels before sending out any more messages.

A marker separates the messages in the channel into those to be included in the snapshot from those not to be recorded in the snapshot.



Chandy-Lamport Algorithm

- Distributed algorithm to capture a consistent global state. Communication channels assumed to be FIFO.
- Uses a *marker* to initiate the algorithm.
- Sending Marker by P:
 - P records its state. For each outgoing channel C, P sends a marker on C with the state info.
- Receiving Marker by Q:
 - If Q has NOT recorded its state: a. Record the state as an empty sequence, SEND marker (use above rule).
 - Else (Q has recorded state before): Record the state of C as sequence of messages received along C, after Q's state was recorded and before Q received the marker.

Chandy-Lamport algorithm

- The algorithm can be initiated by any process by executing the “Marker Sending Rule” by which it records its local state and sends a marker on each outgoing channel.
- A process executes the “Marker Receiving Rule” on receiving a marker. If the process has not yet recorded its local state, it records the state of the channel on which the marker is received as empty and executes the “Marker Sending Rule” to record its local state.
- The algorithm terminates after each process has received a marker on all of its incoming channels.
- All the local snapshots get disseminated to all other processes and all the processes can determine the global state.

Chandy-Lamport algorithm

Time ↓

P: [saves state] ---MARKER--->

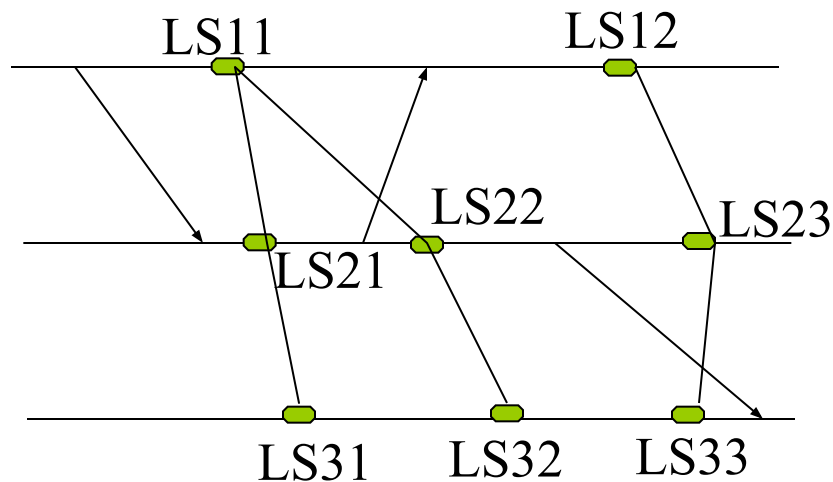
Q: receives MARKER
[saves state]
---MARKER--->

R: receives MARKER
[saves state]
---MARKER--->

P: receives MARKER
(records channel messages)

Recording Global State...

- Identify consistent, inconsistent, and strongly consistent global state.



Recording Global State...

- Identify consistent (LS12, LS23, LS33), and (LS11, LS22, LS32), inconsistent (0), and strongly consistent global state (LS11, LS21, LS31) .

